EXP 4	A PYTHON PROGRAM TO SOLVE AN IPP USING THE BRANCH AND BOUND METHOD

Solving Integer Programming Problems using the Branch and Bound Method

AIM:

To solve any integer programming problem using the Branch and Bound method in Python.

PROCEDURE:

1. Formulate the integer programming problem.
2. Implement the Branch and Bound method in Python.
3. Solve the problem using the implemented method.
4. Verify the solution.

PROBLEM – 1: SOLVE THE FOLLOWING INTEGER PROGRAMMING PROBLEM USING BRANCH AND BOUND

Maximize

$$Z = 4x + 6y$$

Subject to:

- $3x + 2y \leq 18$
- $x + 3y \leq 15$
- $x \leq 7$
- $y \leq 6$
- $x, y \geq 0$ and integers

CODE:

```
import numpy as np

from scipy.optimize import linprog

from queue import Queue

def solve_lp(c, A, b, bounds):
```

```
    return linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='highs')

def branch_and_bound(c, A, b, bounds):
    Q = Queue()
    Q.put((c, A, b, bounds))
    best_solution = None
    best_value = float('-inf')
    while not Q.empty():
        c, A, b, bounds = Q.get()
        res = solve_lp(c, A, b, bounds)
        if res.success:
            solution = res.x
            value = -res.fun
            # Check if integer
            if all(np.floor(solution) == solution):
                if value > best_value:
                    best_value = value
                    best_solution = solution
            else:
                for i in range(len(solution)):
                    if solution[i] != int(solution[i]):
                        lb = list(bounds)
                        ub = list(bounds)
                        lb[i] = (bounds[i][0], np.floor(solution[i]))
                        ub[i] = (np.ceil(solution[i]), bounds[i][1])

                        Q.put((c, A, b, lb))
```

```
                Q.put((c, A, b, ub))
                break

    return best_solution, best_value
# Problem data
c = [-4, -6]   # Maximize → minimize negative
A = [[3, 2], [1, 3], [1, 0], [0, 1]]
b = [18, 15, 7, 6]
bounds = [(0, None), (0, None)]
solution, value = branch_and_bound(c, A, b, bounds)
print("Problem: Max Z = 4x + 6y")
print("Optimal solution:", solution)
print("Optimal value:", value)
```

OUTPUT:

Problem: Max Z = 4x + 6y

Optimal solution: [4. 4.]

Optimal value: 40.0

PROBLEM – 2: SOLVE THE FOLLOWING INTEGER PROGRAMMING (MINIMIZATION) USING BRANCH AND BOUND

Minimize $Z = 5x + 4y$

Subject to:

- $2x + y \geq 8$
- $x + 2y \geq 10$
- $x \leq 6$
- $y \leq 6$
- $x, y \geq 0$ and integers

CODE:

```
import numpy as np
from scipy.optimize import linprog
from queue import Queue
def solve_lp(c, A, b, bounds):
    return linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='highs')
def branch_and_bound(c, A, b, bounds):
    Q = Queue()
    Q.put((c, A, b, bounds))
    best_solution = None
    best_value = float('inf')
    while not Q.empty():
        c, A, b, bounds = Q.get()
        res = solve_lp(c, A, b, bounds)
        if res.success:
            solution = res.x
            value = res.fun
            # Check integer solution
            if all(np.floor(solution) == solution):
                if value < best_value:
                    best_value = value
                    best_solution = solution
            else:
                for i in range(len(solution)):
                    if solution[i] != int(solution[i]):
                        lb = list(bounds)
                        ub = list(bounds)
```

```
                lb[i] = (bounds[i][0], np.floor(solution[i]))
                ub[i] = (np.ceil(solution[i]), bounds[i][1])

                Q.put((c, A, b, lb))
                Q.put((c, A, b, ub))
                break
    return best_solution, best_value
# Convert ≥ constraints to ≤ by multiplying -1
c = [5, 4]
A = [[-2, -1], [-1, -2], [1, 0], [0, 1]]
b = [-8, -10, 6, 6]
bounds = [(0, None), (0, None)]
solution, value = branch_and_bound(c, A, b, bounds)
print("Problem: Min Z = 5x + 4y")
print("Optimal solution:", solution)
print("Optimal value:", value)
```

OUTPUT:

```
Problem: Min Z = 5x + 4y
Optimal solution: [2. 4.]
Optimal value: 26.0
```

RESULT:

Thus, solving any Integer Programming Problem using the Branch and Bound method in Python has been implemented and executed successfully.